

Software types
Generic : developed to be sold to mass customer
Bespoke, custom mode : developed to a single customer based on their specification

Capability Maturity Model Integration (CMMI)

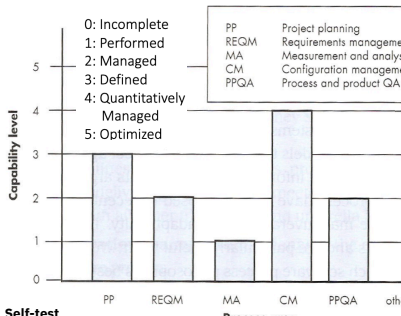
- Software engineering institute (SEI)
- CMMI model is relied on a set of system and software engineering capabilities
- CMMI model represents different levels of process capabilities and maturity

To achieve capabilities
Company should develop process model and make sure it follow the guideline of CMMI.

CMMI representation of process model

- Continuous model : capability
 - Each process area is assessed against specific goal based on capability levels
 - 0. Level 0 : incomplete
 - i. Level 1 : performed
 - ii. Level 2 : managed
 - iii. Level 3 : defined
 - iv. Level 4 : Quantitatively managed
 - v. Level 5 : optimized
- Stage : maturity

CMMI's goal
CMMI defined a set of generic goals and related practice
Each goal corresponds to one of the five capabilities level.
To achieve a particular capability level
Generic goal for that level and generic practices that corresponds to that goal must be achieved.



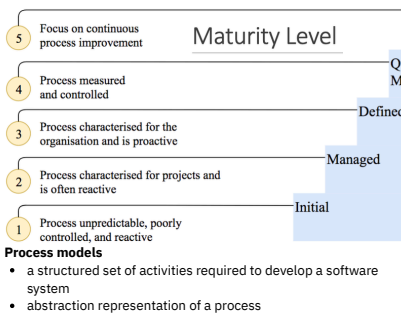
Self-test
The development team starts the project right away by writing the codes. When they need more information, they contact the customer/client to gather/clarify some requirements.

CMMI level self : Performed ✓

The development team uses Git to help them back up and manage their project's codes.

CMMI level self : Performed ✓

Staged model
same process areas, goal, and practices as continuous model same as continuous
Differences : five maturity level, than five capability



Process models

- a structured set of activities required to develop a software system
- abstraction representation of a process

Agile principle
focus on customer satisfaction and early incremental

- Software
- small, highly motivated project team
- minimal software engineering work product
- overall development simplicity

Agile process, the principle

- Hard to predict software requirement, and how it will change, customer's software priority as well
- Design and construction are merged, both should be performed in sequence
- Analysis, design, construction, and testing are not predictable

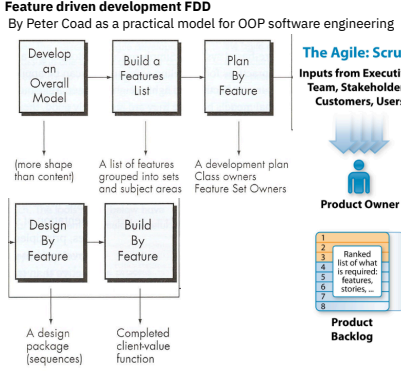
Agile method (AM)
Agile method is a practice-based methodology for effective modeling and documentation of software-based system.

Agile models are better because they are just good, no need for perfection, stupidly wrong

CBSE

- component analysis, see what you have
- requirements modification
- system design with reuse
- development and integration

Feature driven development FDD
By Peter Coad as a practical model for OOP software engineering



Agile software process model
Feb 2001, 17 software developers met at Snowbird, Utah resort to publish Manifesto for Agile software.

- Individuals and interactions
- Working software over comprehensive documentation
- Customer collaboration
- Responding to change

Post-agile

Team structure

- Silo (works individually, low coupling, high cohesion)
- Clear responsibility

Team discipline

- Document oriented
- Each step has a clear definition of document standard
- Timeline is king
- change is bad

Individual discipline

- Make something done but not right
- Please do not change it
- Make it for last, not for change

Treat change as something to be scared of

Agile

Team structure

- One single team, break the silo

Team discipline

- Change is good, we love change
- Eliminate unchangeable module

Individual discipline

- Super clear and have to follow
- Test : code : refactor

Scrum

User stories : A story of what the user wants to happen, idealistic scenario which becomes backlog

Scrum board : not check out, check out, done

Daily meeting (client representative can join)

PM Tool : scrum board (can be in a digital version, backlog being moved through each column)

Sprint burn down chart : a planned time taken to finish project vs reality line, if real line is below ideal line, that means things might be done sooner than expected, opposite situation creates opposite outcome

version control : github and bitbucket

One sprint for certain number of backlog, sprint backlog

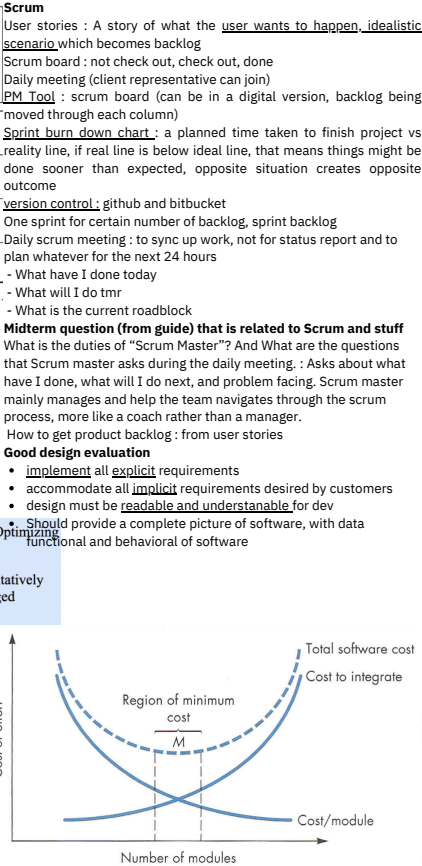
Daily scrum meeting : to sync up work, not for status report and to plan whatever for the next 24 hours

- What have I done today
- What will I do tmr
- What is the current roadblock

Midterm question (from guide) that is related to Scrum and stuff
What is the duties of "Scrum Master"? And What are the questions that Scrum master asks during the daily meeting. : Asks about what have I done, what will I do next, and problem facing. Scrum master mainly manages and help the team navigates through the scrum process, more like a coach rather than a manager.
How to get product backlog : from user stories

Good design evaluation

- implement all explicit requirements
- accommodate all implicit requirements desired by customers
- design must be readable and understandable for dev
- Should provide a complete picture of software, with data functional and behavioral of software



Design model
Customer requirements, business needs, and technical consideration meets

Provides details about

- Software data structure
- architecture
- interfaces
- components that are necessary to implement in the system

this model can be assessed for software quality before

- code is generated, test are conducted, end-users become involved in large number
- this is where software quality is established
- Architecture of the system or product
- Interface that connect the software to :
 - end-users
 - other systems or devices
 - components
- Software components that are used to construct sys

Class-based modelling

Level of abstraction
class attr and methods
Procedural abstraction : sequence of instructions that have specific function, word open lists a long procedural abstraction
data abstraction : naed collection of data that describes a data object, class attribute

Message
information sent to object to trigger methods
signal for method call of action
contains

- procedure name
- information : parameter list

Generalization and inheritance
pros : Abstraction mechanism, may be used to classify entities
reused mechanism at both design and programming level
inheritance graph is a source of organizational knowledge about domains and system

cons, - classes are not self-contained
tendency to reuse inheritance graph during design process (inheritance graph from analysis part)
inheritance graph for analysis, design, implementation, should be separated

Polymorphism & dynamic binding
Polymorphism : same message can be interpreted differently by different classes of objects. Doctor_name and name, Doctor, birth_date and birth_date

Class diagram : Derived: slash (/) | Public + everyone | Protected # same and derived | Private - self-only

Operations : create | read (query) | update
association 1..* 0..3, 5

Aggregation and composition

- aggregation "is a part of" / "has a" white diamond, can exist without it
- composition "is entirely made of", black diamond cannot exist without it, live and die with it

Class diagram, arrow head meaning
Unidirectional, order needs to know the name of the customer it will be sent to, customer is not necessarily have that data to exist. Existence requirement of other class's variable. Order points at customer. Bidirectional is no arrow at all.

Software architecture

- structure or organization of program components (modules)
- Overall structure of software

Modularity & Software cost
separated into separated components

Methods and message
Message from new instance is sent to application, the method of a class will be called and proceeded.

Information hiding
To hide detail of data structure and procedural processing behind a module interface.

Encapsulation & information hiding
Encapsulation is when attributes and methods are packed in one class instead of being global or native to otherclass, information hiding is when getter method

Functional independence
Direct consequence of modularity, abstraction, and information hiding

Cohesion and Coupling
Cohesion, focus on its task
"GOD OBJECT"
Methods and attr similar = high cohesion, good, not similar, low cohesion, bad

- Low cohesion : method is not related to class, utility class, hidden objects and subclasses

Coupling
how much one class knows about others,

- Tight/high coupling : classes tied together
- loose/low coupling : classes only relate to the practical extent, interface is introduced here

High cohesion, loose coupling
Design class
Refine the analysis classes by providing design details.

- User interface classes
- Business domain classes
- Process classes
- Persistent classes
- System classes

Model-view-control & Model-view-view-model
Model view control uses UI as a display only, but Model view view model uses UI to take user input as well.

Well-formed design class

- Complete and sufficient
- Primitive (or Originality)
- High cohesion
- Low Coupling

Data design elements
Create a model of data and/or information that is represented as a high level of abstraction

Architectural design element
From :

- Information** about the application domain for the software to be built
- Specific analysis model elements
- Availability of architectural patterns and styles

Interface design elements

- UI
- External interfaces
- Internal interfaces

Distributed system
Information processing is distributed over several computers rather than confined to a single machine
Very important for enterprise computing system

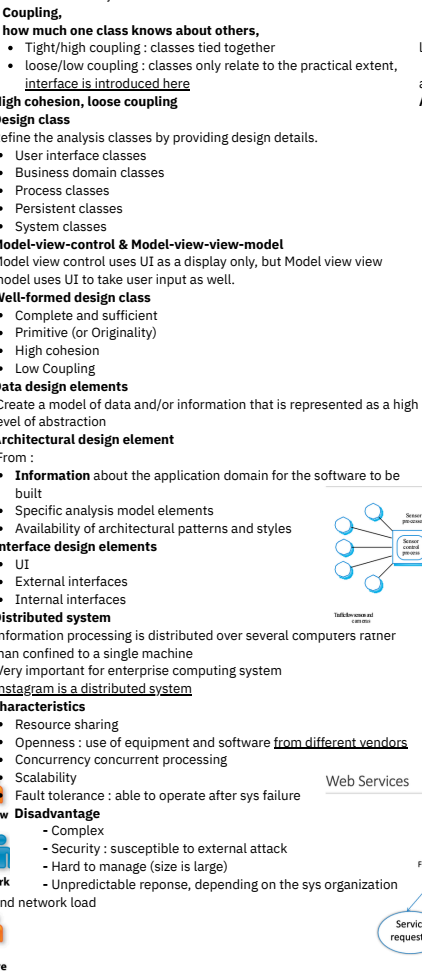
Instagram is a distributed system

characteristics

- Resource sharing
- Openness : use of equipment and software from different vendors
- Concurrency concurrent processing
- Scalability
- Fault tolerance : able to operate after sys failure

Disadvantage

- Complex
- Security : susceptible to external attack
- Hard to manage (size is large)
- Unpredictable reponse, depending on the sys organization and network load



Distri. Sys. Architecture

- Client-server architecture
 - Client and service server are treated differently, permission restriction
 - Clients know servers, servers need not to know client
- Distributed object architectures
 - No distinction between clients and server
 - Any objects can provide and use service from each other
 - Communication happens through object request broker, middleware
 - More complex than C/S system
 - Data mining system

Multi-processor architectures

- Simplest distributed system model
- Composed of multiple processes which can execute on different processors
- This is a base for many large real-time system
- Many processes, different modules, work differently, concurrently

Layered application architecture

- Presentation : presenting result to user
- Application : a function, open bank acc, close acc..
- Data management : DB management

Client --- Server
Thin-client
Presenting --- [application - data management]

- Legacy (app and data are together), computationally-intensive, data-intensive

Fat client
[presenting - application] --- [data management]

- A genuine software product like Excel, data visualization, stable end-user

Three-tier architecture

- Each layer can execute on a separate processor
- Better performance than thin-client, simpler to manage than fat-client
- Scalable as server extension is possible
- Many users

Service-oriented architecture (SOA)
Web available software

- Externally provided service (web service)
- Standard approach to make a reusable component available across the web.

Service standard
Based on agreed XML-based standards so can be provided on any platform and written in any programming language
SOAP - simple object access protocol
WSDL - Web services description language
UDDI - Universal description discovery and integration

Application types

- Data processing application
 - a. Billing systems, payroll systems
 - b. Data is input and processed and output in batches
- Transaction processing application
 - a. E-commerce, reservation
 - b. Query or modify data from DB
- Event processing systems
 - a. Word processors, real-time systems
 - b. System reacts to events, like games, real-time editing system including rapid feedback to user.
- Language processing systems
 - a. Compilers, command interpreters
 - b. Compiler

